

AI-CliniScan: Automated Detection and Localization of Chest X-Ray Abnormalities

Author: Tirumani Surya Siva Shankar

Table of Contents

1. Project Description
 2. Dataset Used
 3. Environment Setup
 4. Data Exploration
 5. Data Preprocessing 5.1. Image Loading & Resizing 5.2. Ground Truth Extraction & YOLO Conversion 5.3. Train/Validation Split 5.4. Data Augmentation Pipeline
 6. Proposed System
 7. Training Pipeline
 8. Conclusion
-

1. Project Description

AI-CliniScan is a deep-learning-based computer vision project designed to automatically detect and localize abnormalities in chest radiographs. The primary goal of this project is to build an automated diagnostic tool that not only identifies whether a thoracic abnormality exists but also draws precise bounding boxes indicating its exact location. This project implements the full machine learning workflow, from rigorous data preprocessing of thousands of medical images to the training and evaluation of a state-of-the-art YOLOv8 object detection model.

2. Dataset Used

The project utilizes the VinBigData Chest X-ray Abnormalities Detection dataset. For this baseline phase, a robust subset of 5,000 images was processed. For each image in the dataset:

- A medical image file is provided.
- A corresponding CSV file contains the ground truth annotations, including the class ID and bounding box coordinates (xmin, ymin, xmax, ymax) for any present abnormalities.

The dataset includes 14 distinct classes of thoracic abnormalities, such as Aortic enlargement, Cardiomegaly, Pleural effusion, and Pulmonary fibrosis. It is widely used for benchmarking computer vision models in healthcare due to its high-quality annotations provided by expert radiologists.

3. Environment Setup

The project was implemented using a hybrid cloud and local environment approach to maximize computational efficiency. The key tools and libraries required include:

- **Kaggle Cloud Environment:** Used for the heavy-lifting of preprocessing 5,000 high-resolution images.
- **VS Code (macOS):** Used for local model training and inference.
- **Ultralytics (YOLOv8) & PyTorch:** The core framework used to build, train, and evaluate the deep learning object detection model.
- **Apple Metal Performance Shaders (MPS):** Utilized via PyTorch (`device='mps'`) to hardware-accelerate model training on Apple Silicon GPUs.
- **OpenCV and Albumentations:** Used for image loading, resizing, and dynamically applying data augmentations.
- **Pandas & NumPy:** For efficient numerical operations, CSV parsing, and tabular data handling.

4. Data Exploration

Before preprocessing, the raw dataset and CSV annotations were explored to ensure structural integrity. The exploration involved:

- Verifying the dataset folder paths within the Kaggle environment.
- Extracting coordinate data from the training CSV to verify bounding box constraints.
- Mapping the 14 numerical class IDs to their respective human-readable abnormality labels.
- This step ensured that the raw coordinates correctly aligned with the anatomical structures in the X-rays before transforming them for the neural network.

5. Data Preprocessing

This stage involves converting raw dataset images and CSV annotations into a format specifically required by the YOLOv8 architecture. The preprocessing pipeline includes the following key steps:

5.1. Image Loading & Resizing Original medical images are often extremely large and vary in aspect ratio. For training efficiency and consistent model input, all 5,000 images were uniformly resized to 512x512 pixels.

- **Purpose:** Reduces GPU memory usage, significantly speeds up training times, and maintains uniform input dimensions for the neural network.

5.2. Ground Truth Extraction & YOLO Conversion The raw CSV contains absolute bounding box coordinates (xmin, ymin, xmax, ymax). YOLO architectures require annotations to be in a specific, normalized `.txt` format.

- **Steps:** Extracted the absolute coordinates, calculated the center X, center Y, width, and height of the bounding box, and normalized these values between 0 and 1 relative to the 512x512 image size.
- **Purpose:** Converts raw dataset annotations into the exact `[class_id x_center y_center width height]` mathematical format required by YOLO.

5.3. Train/Validation Split The 5,000 preprocessed images and their corresponding `.txt` label files were programmatically divided into an 80/20 split.

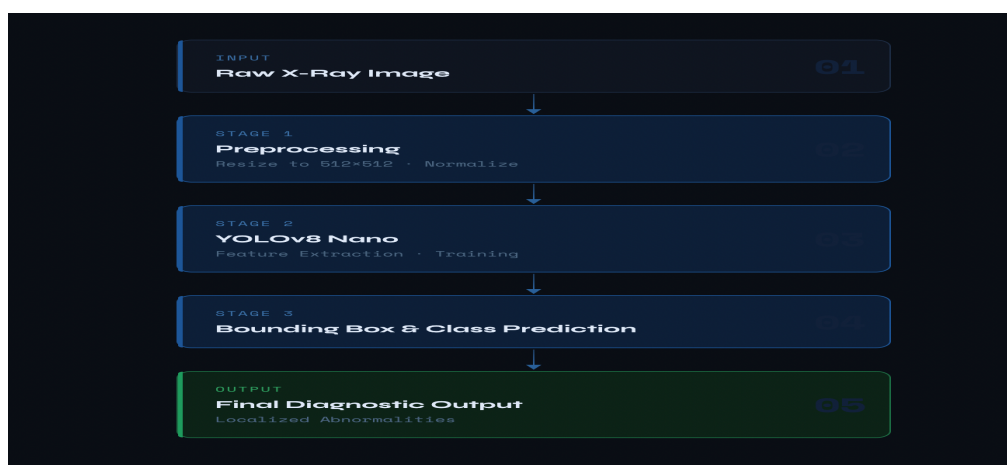
- **Purpose:** Provides 4,000 images for the model to learn from (Training set) and 1,000 unseen images to test the model's accuracy (Validation set).

5.4. Data Augmentation Pipeline To prevent overfitting and make the model more robust to different X-ray conditions, an augmentation pipeline was constructed using the `albumentations` library.

- **Techniques applied:** Horizontal flips, mild rotations, random brightness/contrast adjustments, mild zoom-in (Affine), and elastic deformations.
- **Purpose:** Artificially expands the diversity of the training data, ensuring the model generalizes well to new, real-world clinical images.

6. Proposed System

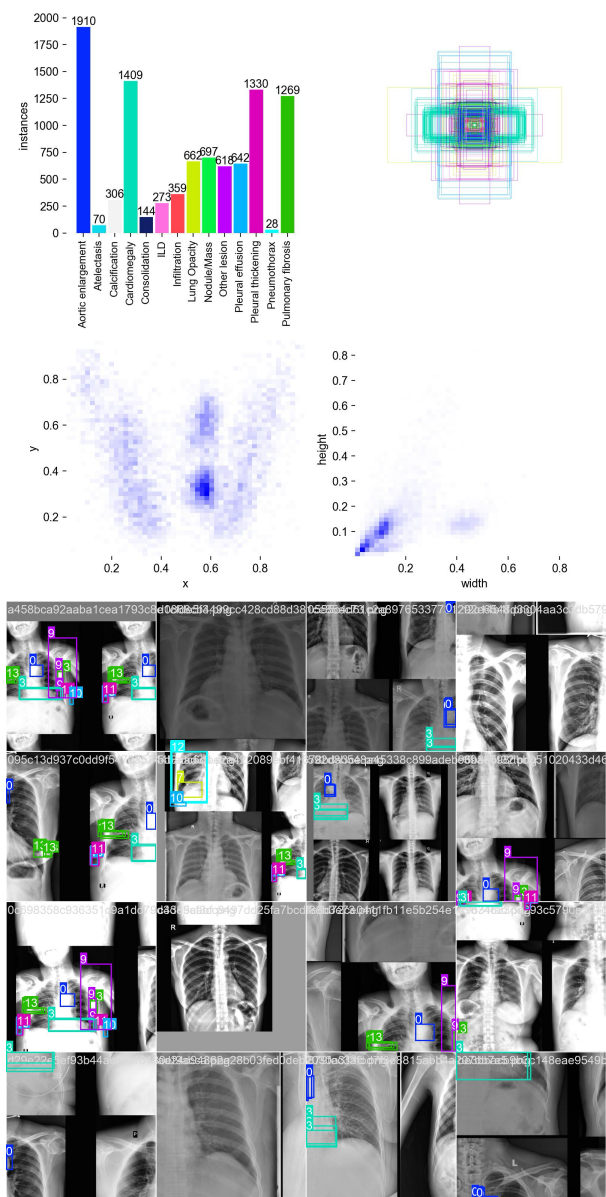
1. **Raw High-Resolution X-Ray:** The system starts with the original chest radiograph containing varying lighting conditions and resolutions.
2. **Preprocessing (Resize & Format):** The image is resized to 512x512, and annotations are converted to normalized YOLO format.
3. **YOLOv8n (Model Training / Inference):** The preprocessed images are passed to the YOLOv8 Nano architecture. The model learns dense spatial features required to identify the edges of lesions and organs.
4. **Bounding Box Prediction:** The model outputs predicted bounding boxes, each accompanied by a class label (e.g., "Cardiomegaly") and a confidence score.
5. **Final Diagnostic Output:** The coordinates are overlaid directly onto the X-ray, providing a visual, localized diagnostic report for the clinician.



7. Training Pipeline

The baseline model was trained locally using the following pipeline configurations:

- **Architecture:** YOLOv8 Nano (`yolo8n.pt`) chosen for its optimized balance of speed and baseline accuracy.
- **Hardware:** Apple Silicon GPU (MPS) for accelerated matrix multiplications.
- **Hyperparameters:** Trained for 15 epochs, utilizing a batch size of 16, and an image size of 512x512.
- **Evaluation:** The model actively generated validation metrics at the end of every epoch, calculating precision, recall, and Mean Average Precision (mAP) across all 14 abnormality classes.



8. Conclusion

The initial phases of the AI-CliniScan project were completed successfully. A robust data pipeline was engineered to process 5,000 high-resolution chest X-rays, systematically converting raw CSV bounding box data into normalized YOLO format while handling rigorous train/validation splitting.

Following preprocessing, a YOLOv8 Nano object detection model was successfully trained over 15 epochs utilizing local GPU acceleration. The model demonstrated the fundamental ability to learn spatial representations of thoracic abnormalities, effectively outputting bounding boxes and confidence scores on unseen validation images. These milestones establish a highly functional, end-to-end baseline architecture that is ready for further optimization and eventual web deployment.

